

Testbed 5G Standalone con Network Slicing e Servizi Verticali

Relazione Tecnica per l'Esame di NCIs

1 Ottobre 2026

1 Infrastruttura di Rete e Configurazione Nodi

Il testbed realizza una rete cellulare **5G Standalone (5G SA)** secondo lo standard 3GPP Release 16, basata sul principio della disaggregazione **CUPS (*Control and User Plane Separation*)**. Tutta l'infrastruttura è configurata per girare interamente in locale su una singola macchina (`localhost`), ricalcando fedelmente il modello delle **Reti Private 5G aziendali (*Private NPN*)**.

La configurazione adotta una singola istanza virtuale con sistema operativo Ubuntu 22.04 Desktop, eseguita a seconda dell'ambiente host tramite UTM oppure VirtualBox. Questa eterogeneità degli strumenti di virtualizzazione non ha impattato sull'ambiente applicativo, poiché in entrambi i casi tutti i moduli funzionali girano concorrentemente all'interno dello stesso sistema operativo guest, realizzando un'architettura single-node centralizzata per l'elaborazione, la gestione delle code e l'instradamento del traffico.

I compiti all'interno della rete sono così suddivisi:

- **Host Control Plane (Open5GS CP):** I moduli del Core comunicano tra loro tramite il bus **SBA (*Service-Based Architecture*)** sfruttando chiamate REST su protocollo **HTTP/2**:
 - **AMF (*Access and Mobility Management Function*):** È il punto di ingresso per il controllo: gestisce chi si collega alla rete, autentica i terminali e ne segue la mobilità. Espone l'interfaccia **N2 (NGAP su SCTP porta 38412)** su cui si attesta l'antenna gNodeB locale (`127.0.0.5:38412`).
 - **SMF (*Session Management Function*):** È il modulo che apre, modifica e chiude materialmente le sessioni dati dei dispositivi. Per gestire al meglio il Network Slicing, abbiamo istanziato **tre SMF separate (smfd, smf2d, smf3d)**, ciascuna dedicata a una specifica slice. Ogni SMF controlla la propria UPF tramite l'interfaccia standard **N4 (protocollo PFCP su UDP porta 8805)**.
 - **UDM e UDR:** Contengono i profili, gli abbonamenti e le chiavi segrete degli utenti registrati, appoggiandosi fisicamente su un'istanza locale del database documentale **MongoDB (mongodb://localhost/open5gs)**.
 - **AUSF:** Lavora a stretto contatto con l'UDM per verificare la sicurezza e autenticare le SIM con l'algoritmo standard 5G-AKA.
 - **NRF e SCP:** L'NRF fa da "rubrica telefonica" interna del core, permettendo ai vari moduli di trovarsi automaticamente all'avvio; l'SCP fa da proxy per instradare e smistare i messaggi HTTP/2 tra i microservizi (secondo la specifica 3GPP TS 29.500).
 - **PCF e BSF:** Il PCF è il "cervello" delle regole di traffico: stabilisce le priorità di rete, i parametri QoS e i limiti massimi di velocità (Session-AMBR) per ciascun abbonato.
 - **NSSF:** Si occupa di analizzare le richieste dei terminali e di indirizzarli verso la slice di rete corretta.
- **Accesso Radio e Terminali (UERANSIM):**
 - **UERANSIM nr-gnb:** Agisce come una vera e propria antenna 5G (il gNodeB). Da un lato parla con l'AMF via SCTP (interfaccia N2) per la segnalazione, dall'altro inoltra i pacchetti dati verso le UPF incapsulandoli in tunnel **GTP-U su UDP porta 2152 (interfaccia N3)**. Essendo un emulatore software accademico, scambia i protocolli radio (RRC e NAS)

con i terminali direttamente tramite socket UDP locali (porta 4997): questo ci permette di testare con precisione chirurgica tutta la logica 3GPP senza le complicazioni e le interferenze tipiche della propagazione elettromagnetica su frequenze reali (come le bande a 700 MHz o 3.6 GHz).

- **UERANSIM nr-ue:** Emula i dispositivi degli utenti (smartphone, sensori IoT). Ciascun UE possiede una propria identità IMSI, chiavi crittografiche virtuali e gestisce l’aggancio alla rete e l’apertura delle sessioni dati.
- **User Plane Function (Open5GS UPF 1, 2, 3):** Abbiamo configurato **tre istanze separate di UPF** (upfd, upf2d, upf3d), una per ciascuna slice. Le UPF funzionano in pratica come router di uscita: ricevono i tunnel GTP-U dall’antenna, tolgono l’incapsulamento e instradano i dati grezzi dei dispositivi verso l’esterno attraverso tre schede virtuali di rete kernel TUN (ogstun per la slice 1, ogstun2 per la slice 2, ogstun3 per la slice 3), applicando il classico NAT per farli navigare.

1.1 Modello di QoS 3GPP: 5QI, ARP e Parametri di Profilo

Seguendo lo standard **3GPP TS 23.501**, la qualità del servizio (QoS) di una sessione PDU è definita da alcuni parametri fondamentali:

- **5QI (5G QoS Identifier):** Un valore numerico da 1 a 255 che indica alla rete come trattare i pacchetti. Divide il traffico in due grandi famiglie:
 - *GBR (Guaranteed Bit Rate):* La rete riserva una banda minima garantita fissa; è indispensabile per servizi in tempo reale (come le chiamate voce) dove i ritardi renderebbero la conversazione incomprensibile.
 - *Non-GBR:* Traffico a “miglior sforzo” (best-effort); la velocità si adatta alla congestione della rete (tipico del download di file o del web).

I valori più bassi (es. 5QI 1–3) identificano flussi ad altissima priorità e minima latenza, mentre valori più alti (es. 5QI 6–9 o 70) tollerano ritardi maggiori e vengono accodati se la rete è carica.

- **ARP (Allocation and Retention Priority):** Assegna una priorità da 1 a 15 (dove 1 è la massima) e stabilisce le regole di precedenza (*pre-emption*) per decidere quali flussi preservare e quali sacrificare se la rete dovesse saturarsi. Nel nostro progetto abbiamo assegnato valore 1 a URLLC, 4 a eMBB e 12 a MIoT.
- **Session-AMBR (Aggregate Maximum Bit Rate):** È il “tetto” massimo di velocità complessiva (sia in download che in upload) consentito per la sessione. Nel profilo del Core abbiamo concesso fino a 1 Gbps alla slice eMBB, limitando invece a 1 Mbps la slice MIoT.

2 Servizi Verticali Integrati nel Data Network (DN)

Per validare l’infrastruttura con carichi di lavoro realistici e diversi tra loro, abbiamo collegato alla rete dati quattro servizi applicativi reali, insieme a server di misura dedicati:

1. **Nextcloud (Storage Cloud – Slice eMBB):** Un’istanza locale esposta su porta TCP 80 usata per caricare e scaricare file pesanti, con l’obiettivo di testare il throughput di picco e la stabilità dei trasferimenti HTTP attraverso il tunnel 5G.
2. **Asterisk PBX + Twinkle (VoIP – Slice URLLC):** Un centralino telefonico PBX Asterisk v18 (attivo in locale sulla porta UDP 5060) e flussi voce real-time RTP (porte UDP 10000–20000). Abbiamo configurato due interni (1001 e 1002) per simulare una telefonata vocale tra due telefoni software Twinkle e misurare con precisione tempi di ritardo e jitter. Questo servizio risiede localmente sulla macchina comportandosi a tutti gli effetti come un’applicazione **MEC (Multi-access Edge Computing) a breakout locale**, azzerando i tempi di transito geografici.

3. **Mosquitto MQTT (IoT Telemetria – Slice MIoT):** Un broker di messaggistica Publish/Subscribe leggero (porta TCP 1883) per raccogliere dati simulati di sensori (temperatura, umidità), utile per analizzare l'impatto dell'incapsulamento della rete cellulare sui pacchetti molto piccoli.
4. **Server Multi-Porta iperf3:** Tre istanze del benchmark iperf3 sempre attive in background tramite servizio di sistema (`iperf3-servers.service`) sulle porte 5201, 5202 e 5203, dedicate a misurare scientificamente banda e latenza su ciascuna delle tre slice.

3 Configurazione del Network Slicing e Provisioning UE

La rete è partizionata su 3 Network Slice distinte secondo lo standard 3GPP TS 23.501, configurate con i parametri riassunti nella Tabella 1.

Tabella 1: Parametri di Configurazione e Prestazioni delle 3 Network Slice

SST	Nome Slice	5QI	ARP	Banda AMBR (Core)	Limitazione Kernel TC	Interfaccia TUN
1	eMBB	6	4	DL: 1 Gbps / UL: 100 Mbps	Canale un-throttled	ogstun (10.45.0.0/16)
2	URLLC	1	1	DL: 100 Mbps / UL: 100 Mbps	TBF rate 100mbit burst 32k	ogstun2 (10.46.0.0/16)
3	MIoT	70	12	DL: 1 Mbps / UL: 512 Kbps	TBF rate 1mbit burst 32k	ogstun3 (10.47.0.0/16)

3.1 Provisioning del Subscriber su MongoDB

L'iscrizione e la configurazione degli abbonati avvengono tramite il database MongoDB locale, i parametri di sicurezza 5G-AKA (la chiave segreta K , l'operatore OP_c , il campo $AMF = 8000_{16}$ e il contatore SQN), associando in modo rigido all'identificativo del terminale (IMSI es. 999700000000001 e 999700000000002) le slice a cui ha diritto, con relativo DNN ("internet"), priorità ARP e limiti di banda AMBR.

3.2 Segmentazione nei Network Namespaces

Per evitare che i terminali simulati facciano confusione con le rotte della macchina principale, UERANSIM sfrutta i **Network Namespace di Linux**. In pratica, ogni volta che un terminale avvia una sessione dati, il sistema crea una "stanza chiusa" virtuale per la rete, con una propria scheda `uesimtun0` e una propria tabella di routing:

```
# Esempio di namespace:
sudo ip netns exec ueransim-9997000000000001-internet-psi1 ip addr show uesimtun0
```

Lanciando qualsiasi comando preceduto da `sudo ip netns exec <nome_namespace>`, il traffico di quel comando viene forzato a passare al 100% all'interno del tunnel della slice corrispondente.

4 Casi di Test

4.1 Test 1: UE Non Registrato in MongoDB (Rogue UE Reject)

Procedura: Abbiamo simulato il tentativo di connessione di un dispositivo non autorizzato (*rogue UE*). Abbiamo creato un file di configurazione con credenziali casuali e un IMSI non censito in MongoDB (`imsi-999700000099999`). All'avvio, il terminale ha inviato la richiesta di registrazione (*Registration Request*) all'antenna gNodeB, che l'ha passata all'AMF via interfaccia N2.

Esito Rilevato: Non appena l'AMF ha chiesto i vettori di autenticazione all'UDM, quest'ultimo ha

risposto con codice 404 Not Found (abbonato inesistente). L'AMF ha stroncato subito la connessione inviando al terminale un pacchetto NAS di Registration Reject con causale standard 5GMM Cause #3 (Illegal UE / FIVEG_SERVICES_NOT_ALLOWED). L'emulatore si è fermato all'istante rilasciando il canale radio, confermando che nessun dispositivo privo di SIM registrata può consumare risorse della rete Core.

4.2 Test 2: Tentativo di Accesso con Slice Non Autorizzata

Procedura: Abbiamo utilizzato un terminale con SIM valida e registrata, ma modificando il suo file di avvio per forzare la richiesta di adesione alla **Slice 4 (SST = 4)**, un tipo di slice non abilitato nel profilo dell'utente e non configurato per la rete "internet".

Esito Rilevato: La prima fase di autenticazione 5G-AKA è andata a buon fine perché l'identità dell'utente era riconosciuta. Quando però il dispositivo ha provato ad aprire la connessione dati (PDU Session Establishment Request) verso la slice 4, l'AMF ha confrontato la richiesta con l'elenco delle slice autorizzate e ha rigettato la sessione con errore NAS [DNN_NOT_SUPPORTED_OR_NOT_SUBSCRIBED]. Sul sistema non è stata creata alcuna scheda virtuale né alcun tunnel GTP-U, a riprova che la separazione tra le slice è rigorosa anche a livello di segnalazione.

4.3 Test 4: Chiamata VoIP con Asterisk e Twinkle (Slice URLLC)

Procedura: Abbiamo associato due softphone Twinkle ai namespace URLLC di UE1 ed UE2. I client si sono registrati via SIP (porta UDP 5060) al PBX Asterisk locale e hanno avviato una chiamata vocale reciproca instradata sulla UPF 2 (ogstun2).

```
# Avvio softphone UE1 (interno 1001) nel namespace URLLC
sudo ip netns exec ueransim-9997000000000001-internet-psi2 \
  sudo -u ubuntu twinkle -c -f ue1.cfg

# Avvio softphone UE2 (interno 1002) nel namespace URLLC
sudo ip netns exec ueransim-9997000000000002-internet-psi2 \
  sudo -u ubuntu env HOME=/home/ubuntu/ue2_home twinkle -c -f ue2.cfg

# Dalla console di UE1 viene composto il numero: call 1002
```

Esito Rilevato: Il setup della sessione (INVITE – 200 OK) è avvenuto correttamente. Il canale audio RTP ha mantenuto un RTT di **1.1 ms**, e **zero pacchetti persi**, rispettando i vincoli di priorità della classe 5QI 1.

4.4 Test 5: Trasferimento File tramite Nextcloud (Slice eMBB)

Procedura: Dal namespace di UE1 abbiamo eseguito l'upload WebDAV/HTTP (porta TCP 80) di un archivio dati verso Nextcloud, completandone poi il download dal namespace separato di UE2 per verificare la banda e l'integrità del trasferimento attraverso la scheda ogstun.

```
# Upload del file di test dal namespace UE1 (Slice eMBB)
sudo ip netns exec ueransim-9997000000000001-internet-psi1 \
  curl -s -u "user:password" -T test_file.zip http://10.45.0.1/remote.
php/dav/files/user/

# Download e verifica dal namespace UE2 (Slice eMBB)
sudo ip netns exec ueransim-9997000000000002-internet-psi1 \
  curl --fail --show-error -u "user2:password" -O http://10.45.0.1/remote.
php/dav/files/user2/test_file.zip
```

Esito Rilevato: Sia l'upload che il download si sono conclusi senza errori (checksum SHA-256 identico). La UPF 1 ha gestito la concorrenza dei due flussi mantenendo un transfer rate elevato, a conferma della capacità della slice di sostenere trasferimenti massivi simultanei.

4.5 Test 6: Messaggistica e Telemetria con Mosquitto MQTT (Slice MIoT)

Procedura: Nel namespace della slice MIoT (SST = 3, 5QI = 70), un client MQTT ha trasmesso periodicamente stringhe JSON con telemetrie simulate verso il broker locale Mosquitto (porta TCP 1883), mentre un secondo client restava in ascolto sul topic dedicato.

```
# Subscriber in ascolto sul topic (eseguibile sull'host o nel namespace UE2)
mosquitto_sub -h 10.47.0.1 -p 1883 -t "sensori/telemetria" -v

# Invio periodico delle letture dal namespace UE1 (Slice MIoT)
sudo ip netns exec ueransim-999700000000001-internet-psi3 \
  mosquitto_pub -h 10.47.0.1 -p 1883 -t "sensori/telemetria" -m '{"temp": 22.4,
"umidita": 60}'
```

Esito Rilevato: Il broker ha smistato le letture con latenza inferiore a **0.5 ms**. L'ispezione con Wireshark ha confermato che su carichi così ridotti l'overhead degli header di trasporto (GTP-U, UDP e IP) incide in misura percentuale significativa rispetto al carico utile (payload), comportamento caratteristico e previsto per le trasmissioni Massive IoT.

4.6 Test 7: Misure di Throughput e Rispetto dei Parametri AMBR con iperf3

Per verificare con precisione se i tetti massimi di velocità (**Session-AMBR**) e le code di shaping del kernel vengano realmente rispettati sul campo, abbiamo eseguito una sessione di test con **iperf3** direttamente dall'interno dei namespace dei terminali verso i gateway delle rispettive UPF.

Tabella 2: Risultati Sperimentali delle Misure iperf3 sulle 3 Network Slice

Slice 5G	S-NSSAI	Gateway Target	Parametri AMBR	Bitrate UL	Bitrate DL	Esito AMBR
Slice 1 (eMBB)	SST: 1	10.45.0.1:5202	DL: 1 Gbps UL: 100 Mbps	100 Mbits/s	225 Mbits/s	Rispettato
Slice 2 (URLLC)	SST: 2	10.46.0.1:5203	DL: 100 Mbps UL: 100 Mbps	93.7 Mbits/s	93.2 Mbits/s	Rispettato
Slice 3 (MIoT)	SST: 3	10.47.0.1:5201	DL: 1 Mbps UL: 512 Kbps	604 Kbits/s	978 Kbits/s	Rispettato

Dove sono configurati i limiti AMBR:

- **Profilo Utente su MongoDB (open5gs.subscribers):** Nel database locale dell'UDM, all'interno del record BSON di ciascun IMSI, per ogni slice autorizzata (sst: 1, 2, 3) sono specificati i campi `ambr.downlink` e `ambr.uplink`.
- **Policy del Core (/etc/open5gs/pcf.yaml):** All'interno della sezione `policy`, per ciascuna slice associata alla DNN "internet", sono definiti i massimali 3GPP:
 - **Slice 1 (eMBB):** Downlink 1 Gbps (unit: 3), Uplink 100 Mbps (unit: 2).
 - **Slice 2 (URLLC):** Downlink 100 Mbps, Uplink 100 Mbps (unit: 2).
 - **Slice 3 (MIoT):** Downlink 1 Mbps (unit: 2), Uplink 512 Kbps (unit: 1).